

СРСП: Решение задач на вложенные условия, сложные логические выражения и оператор switch

Дисциплина: C++ • 2 курс • специальность «Программное обеспечение»

Ф.И.О. студента: Галеева Элеонора Максимовна

Группа: 25 РПО

Дата выполнения: 23.09.26.

Оценка: _____

Задание 1. Доступ к функции приложения

Логическое выражение: `isAuthorized && !accountBlocked && (hasSubscription || isTrial)`

Исходный код программы:

```
#include <iostream>

using namespace std;

int main() {

    bool isAuthorized, hasSubscription, isTrial, accountBlocked;

    cout << "Enter isAuthorized (1/0), hasSubscription (1/0), isTrial (1/0), accountBlocked (1/0): ";

    cin >> isAuthorized >> hasSubscription >> isTrial >> accountBlocked;

    if (isAuthorized && !accountBlocked && (hasSubscription || isTrial)) {

        cout << "Access granted" << endl;

    } else {

        cout << "Access denied" << endl;

    }

}
```

```
}  
  
return 0;  
  
}
```

- **Объяснение:** Оператор `&&` (и) гарантирует, что пользователь обязательно авторизован и аккаунт не заблокирован. `!` (не) инвертирует статус блокировки. Оператор `||` (или) позволяет предоставить доступ либо при наличии подписки, либо во время пробного периода.

Задание 2. Проверка данных перед регистрацией

Исходный код программы:

```
#include <iostream>  
  
using namespace std;  
  
int main() {  
  
    int loginLength, passwordLength;  
  
    bool hasSpecialSymbol;  
  
    cout << "Enter login length, password length, has special symbol (1/0): ";  
  
    cin >> loginLength >> passwordLength >> hasSpecialSymbol;  
  
    if (loginLength >= 5 && loginLength <= 15) {  
  
        if (passwordLength >= 8) {  
  
            if (hasSpecialSymbol) {  
  
                cout << "Registration successful" << endl;  
  
            } else {  
  
                cout << "Special symbol required" << endl;  
  
            }  
  
        }  
  
    }  
  
}
```

```

    }

    } else {

        cout << "Password is too short" << endl;

    }

    } else {

        cout << "Login is too short or too long" << endl;

    }

    return 0;

}

```

- **Ответ на вопрос:** Несколько последовательных вложенных проверок удобны тем, что программа сразу выявляет первую конкретную ошибку пользователя и прекращает дальнейшую проверку, выдавая точное сообщение, а не запутывая общим условием.

Задание 3. Проверка версии программы

Логическое выражение: `(majorVersion > 2 || (majorVersion == 2 && minorVersion >= 5)) && (majorVersion < 4 || (majorVersion == 4 && minorVersion <= 3))`

Исходный код программы:

```

#include <iostream>

using namespace std;

int main() {

    int major, minor;

    char dot;

```

```

cout << "Enter version (e.g. 3.8): ";

cin >> major >> dot >> minor;

if ((major > 2 || (major == 2 && minor >= 5)) && (major < 4 || (major == 4 && minor <= 3))) {

    cout << "Supported" << endl;

} else {

    cout << "Unsupported" << endl;

}

return 0;

}

```

Задание 4. Проверка файла перед загрузкой

Исходный код программы:

```

#include <iostream>

using namespace std;

int main() {

    int sizeMB;

    bool correctExtension, containsVirus;

    cout << "Enter size MB, correctExtension (1/0), containsVirus (1/0): ";

    cin >> sizeMB >> correctExtension >> containsVirus;

    if (containsVirus) {

        cout << "Dangerous file" << endl;

    } else if (sizeMB > 25) {

```

```
        cout << "File is too large" << endl;

    } else if (!correctExtension) {

        cout << "Unsupported file type" << endl;

    } else {

        cout << "Upload allowed" << endl;

    }

    return 0;

}
```

Задание 5. Обработка HTTP-кодов через switch

Исходный код программы:

```
#include <iostream>

using namespace std;

int main() {

    int code;

    cout << "Enter HTTP code: ";

    cin >> code;

    switch (code) {

        case 200: cout << "OK"; break;

        case 201: cout << "Created"; break;

        case 400: cout << "Bad Request"; break;
```

```
case 401: cout << "Unauthorized"; break;

case 404: cout << "Not Found"; break;

case 500: cout << "Server Error"; break;

default: cout << "Unknown status";

}

cout << endl;

return 0;

}
```

- **Ответ на вопрос:** Оператор `switch` идеален для дискретных точечных значений (кодов ответа), делая код чище и читаемее по сравнению с громоздкой цепочкой `if-else` для каждого числа.
-

Задание 6. Обработка команды Git

Исходный код программы:

```
#include <iostream>

using namespace std;

int main() {

    char cmd;

    cout << "Enter Git command (c, p, l, s): ";

    cin >> cmd;

    switch (cmd) {

        case 'c':

            case 'C': cout << "commit"; break;
```

```
    case 'p':

    case 'P': cout << "push"; break;

    case 'l':

    case 'L': cout << "pull"; break;

    case 's':

    case 'S': cout << "status"; break;

    default: cout << "Unknown Git command";

}

cout << endl;

return 0;

}
```

Задание 7. Режим сборки проекта

Исходный код программы:

```
#include <iostream>

using namespace std;

int main() {

    int mode;

    bool testsPassed, compilerErrors;

    cout << "Enter mode (1-Debug, 2-Release, 3-Test): ";

    cin >> mode;
```

```
cout << "Enter compilerErrors (1/0) and testsPassed (1/0): ";

cin >> compilerErrors >> testsPassed;

switch (mode) {

    case 1:

        if (!compilerErrors) cout << "Debug build allowed";

        else cout << "Build failed due to compiler errors";

        break;

    case 2:

        if (!compilerErrors && testsPassed) cout << "Release build allowed";

        else cout << "Release denied: check errors and tests";

        break;

    case 3:

        if (!compilerErrors) cout << "Test build allowed (tests result ignored)";

        else cout << "Build failed due to compiler errors";

        break;

    default:

        cout << "Invalid mode";

}

cout << endl;

return 0;

}
```

Задание 8. Определите результат программы

1. Что выведет программа? `Full access`
 2. Что будет при `level = 1`? `Limited access`
 3. Что будет при `verified = false`? `Limited access`
 4. Что будет при `banned = true`? `Blocked`
 5. Изменится ли что-нибудь, если записать `banned == false` вместо `!banned`?
Нет, логика работы полностью сохранится.
-

Задание 9. Найдите ошибку в логике

1. Правильно ли работает условие? Нет, некорректно.
 2. Найдите значение `size`, которое покажет ошибку: Любое число (например, `0` или `60`), так как из-за оператора `||` условие истинно вообще для всех чисел.
 3. Исправьте условие: `if (size >= 1 && size <= 50)`
 4. Объясните, почему нужен именно выбранный логический оператор: Нужен оператор `&&` (и), чтобы число одновременно удовлетворяло нижней и верхней границам диапазона.
-

Задание 10. Итоговая практическая задача: система проверки Pull Request

Исходный код программы:

```
#include <iostream>

using namespace std;

int main() {

    int branchType;

    bool buildPassed, testsPassed, reviewApproved, hasConflicts;

    cout << "Enter branch type (1-feature, 2-bugfix, 3-hotfix): ";

    cin >> branchType;

    cout << "Enter buildPassed, testsPassed, reviewApproved, hasConflicts (1/0): ";
```

```
cin >> buildPassed >> testsPassed >> reviewApproved >> hasConflicts;
```

```
switch (branchType) {
```

```
    case 1:
```

```
        cout << "Branch: feature\n";
```

```
        if (buildPassed && testsPassed && reviewApproved && !hasConflicts) {
```

```
            cout << "Pull Request accepted";
```

```
        } else {
```

```
            cout << "Pull Request rejected. Reason: ";
```

```
            if (hasConflicts) cout << "conflicts detected";
```

```
            else cout << "missing requirements (build, tests or review)";
```

```
        }
```

```
        break;
```

```
    case 2:
```

```
        cout << "Branch: bugfix\n";
```

```
        if (buildPassed && testsPassed && !hasConflicts) {
```

```
            cout << "Pull Request accepted";
```

```
        } else {
```

```
            cout << "Pull Request rejected. Reason: ";
```

```
            if (hasConflicts) cout << "conflicts detected";
```

```
            else cout << "build or tests failed";
```

```
        }
```

```

        break;

    case 3:

        cout << "Branch: hotfix\n";

        if (buildPassed && reviewApproved && !hasConflicts) {

            cout << "Pull Request accepted";

        } else {

            cout << "Pull Request rejected. Reason: ";

            if (hasConflicts) cout << "conflicts detected";

            else cout << "build failed or review missing";

        }

        break;

    default:

        cout << "Invalid branch type error";

    }

    cout << endl;

    return 0;

}

```

Таблица тестирования (5 тестов):

№	Входные данные (Тип, Build, Tests, Review, Conflicts)	Ожидаемый результат	Фактический результат
1	1, 1, 1, 1, 0	Pull Request accepted	Pull Request accepted

№	Входные данные (Тип, Build, Tests, Review, Conflicts)	Ожидаемый результат	Фактический результат
2	1, 1, 0, 1, 0	Pull Request rejected	Pull Request rejected
3	2, 1, 1, 0, 0	Pull Request accepted	Pull Request accepted
4	3, 1, 0, 1, 1	Pull Request rejected (conflicts)	Pull Request rejected (conflicts)
5	5, 1, 1, 1, 0	Invalid branch type error	Invalid branch type error

Краткий вывод по работе

В ходе выполнения СРСП были изучены и применены на практике вложенные условные конструкции `if/else`, сложные составные логические выражения с операторами `&&`, `||`, `!` и разветвленный оператор выбора `switch`. На примере проверки версий файлов и системы валидации Pull Request закрепились навыки проектирования логики программного обеспечения с учетом различных бизнес-правил и обработки некорректного ввода данных.